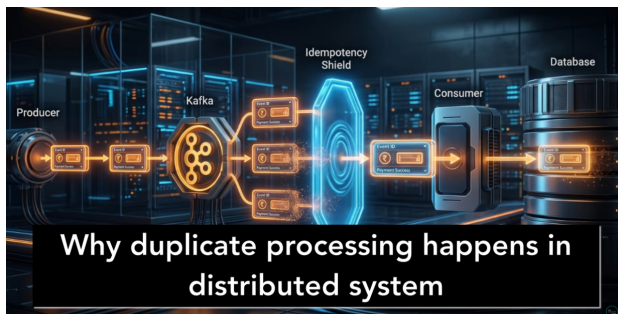
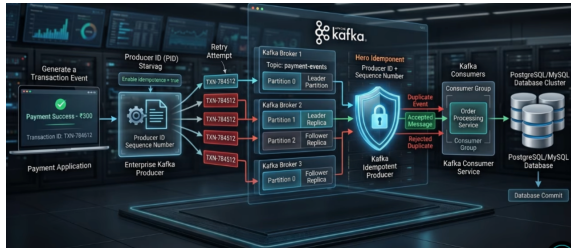
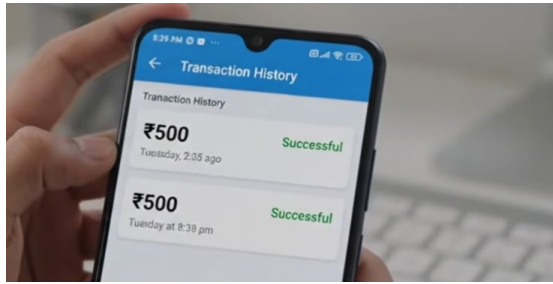
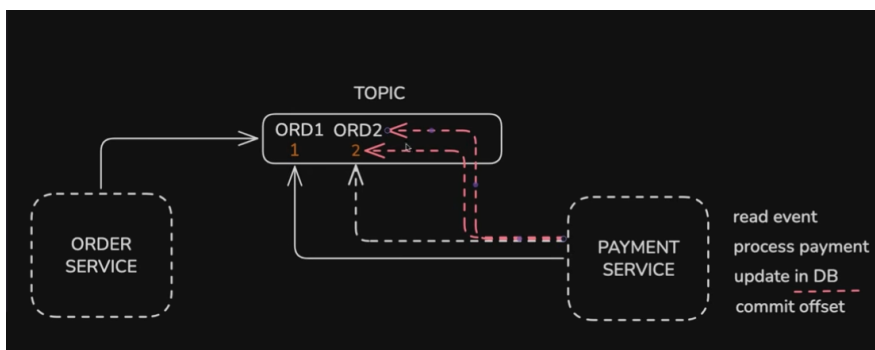


Kafka duplicate transaction



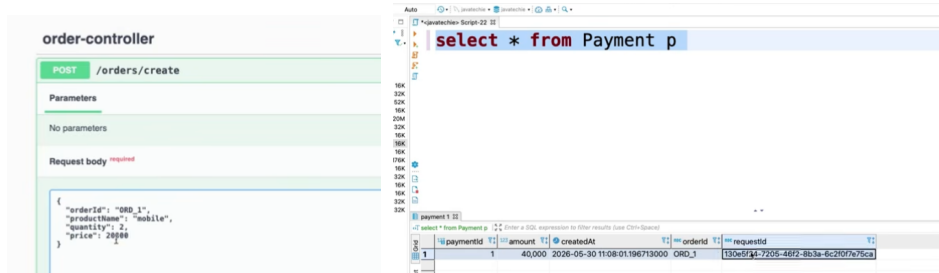
Something went wrong in the distributed system after DB update

Unless offset commit received, kafka retry to read same event again and again



Kafka consumer provides at least one delivery.

Happy Path



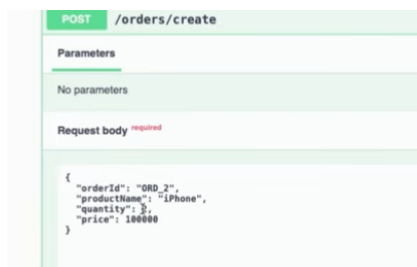
Before commit – in the consumer let us throw runtime exception

```
paymentService.processPayment(orderDto);
log.info("Payment processed for Order ID: {}", orderId);

if(true){
    * throw new RuntimeException("Simulated processing error for Order ID: " + orderId);
}

// Acknowledge the message after successful processing
acknowledgment.acknowledge();
```

Now let us see failure scenario.



producer side no error

```
InfoParser      : Kafka startTimeMs: 1780119480611
startData      : [Producer clientId=order-service-producer-1] Cluster ID: Uzve8lpKwQm0KjbnrydPMC
zationManager  : [Producer clientId=order-service-producer-1] ProducerId set to 0 with epoch 0
```

consumer side error

```

    at org.springframework.kafka.listener.kafkaMessageListenerContainer$ListenerConsumer.do
Caused by: java.lang.RuntimeException: Create breakpoint: Simulated processing error for Order ID: 0
    at com.javatechie.listener.OrderConsumer.processOrder(OrderConsumer.java:34) ~[classes/:na]
    at org.springframework.messaging.handler.invocation.InvocableHandlerMethod.doInvoke(Invocabl
    at org.springframework.kafka.listener.adapter.KotlinAwareInvocableHandlerMethod.doInvoke(Kot
    at org.springframework.messaging.handler.invocation.InvocableHandlerMethod.invoke(Invocable
    at org.springframework.kafka.listener.adapter.HandlerAdapter.invoke(HandlerAdapter.java:80)
    at org.springframework.kafka.listener.adapter.MessagingMessageListenerAdapter.invokeHandler(
    > <4 folded frames>

```

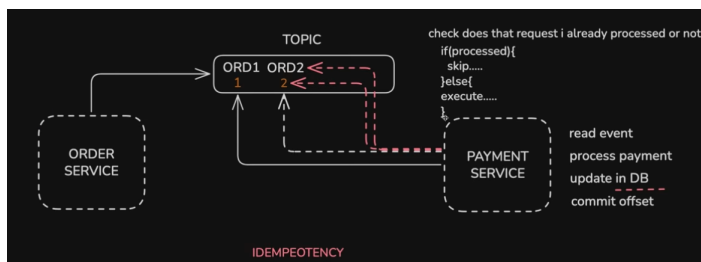
By default, it will try 10 time retry

```
Order ID: ORD_2
Execution[interval=0, currentAttempts=10, maxAttempts=9] exhausted for ORDER_TOPIC-2@1
```

see duplicate transaction

paymentid	amount	createdAt	orderid	requestid
1	40,000	2026-05-30 11:08:01.196713000	ORD_1	130e5f24-7205-46f2-8b3a-6c2f0f7e75ca
2	200,000	2026-05-30 11:12:00.354288000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
3	200,000	2026-05-30 11:12:00.849623000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
4	200,000	2026-05-30 11:12:01.355786000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
5	200,000	2026-05-30 11:12:01.859631000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
6	200,000	2026-05-30 11:12:02.364544000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
7	200,000	2026-05-30 11:12:02.875774000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
8	200,000	2026-05-30 11:12:03.379255000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
9	200,000	2026-05-30 11:12:03.884010000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
10	200,000	2026-05-30 11:12:04.390619000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9

Now let us apply idempotency - check does the request already processed or not.



We can also maintain different idempotency table. / we can also apply unique id constraint for the payment table

paymentid	amount	createdAt	orderid	requestid
1	40,000	2026-05-30 11:08:01.196713000	ORD_1	130e5f24-7205-46f2-8b3a-6c2f0f7e75ca
2	200,000	2026-05-30 11:12:00.354288000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
3	200,000	2026-05-30 11:12:00.849623000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
4	200,000	2026-05-30 11:12:01.355786000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9
5	200,000	2026-05-30 11:12:01.859631000	ORD_2	6d2c3093-d3a3-4f21-b650-08d75bcace9


```
import java.time.LocalDateTime;

@Entity
@Data
@AllArgsConstructor
@NoArgsConstructor
@Table(uniqueConstraints = {@UniqueConstraint(columnNames = {"requestId"})})
public class Payment {

    @Id
    @GeneratedValue
    private Long paymentId;
```

If we apply unique constraint we will get DataIntegrityViolationException so the following below coding

```
// check requestId already exist in db or not - idempotency check
try {
    paymentService.processPayment(orderDto);
    log.info("Payment processed for Order ID: {}", orderId);
} catch (DataIntegrityViolationException ex) {
    log.warn("Duplicate payment attempt detected for Order ID: {}. Skipping process", orderId);
    acknowledgment.acknowledge(); // Acknowledge the message to prevent reprocessing
    return;
}
```

Another way in the consumer, we can do fetch and check request id already present or not if yes just return. Apply manual acknowledgement before return statement

```
@KafkaListener(topics = ORDER_TOPIC, groupId = "payment_group") no usages
public void processOrder(OrderDto orderDto, Acknowledgment acknowledgment) {
    String orderId = orderDto.getOrderId();
    log.info("Received order event for Order ID: {}", orderId);

    // check requestId already exist in db or not - idempotency check
    if (paymentService.isRequestIdExists(orderDto.getRequestId())) {
        log.warn("Order with Request ID: {} already exists.", orderDto.getRequestId());
        return;
    }
}
```

You will see duplicate entry message now.

```
OrderConsumer : Payment processed for Order ID: ORD_3
Consumer : [Consumer clientId=consumer-payment_group-1, groupId=payment_group] Seeking to
ConsumerContainer : Record in retry and not yet recovered
OrderConsumer : Received order event for Order ID: ORD_3

Error : HHH000247: ErrorCode: 1062, SQLState: 23000
Error : Duplicate entry 'c9a256c8-892f-4d6e-b988-2e7f0b1c3fc0' for key 'payment.UKablj
OrderConsumer : Duplicate payment attempt detected for Order ID: ORD_3. Skipping processing.
```

32K

payment 1

select * from Payment p

	paymentid	amount	createdAt	orderId	requestid
1	1	10,000	2026-05-30 11:27:05.904250000	ORD_3	c9a256c8-892f-4d6e-b988-2e7f0b1c3fc0